MATH20008h.01 最优化方法 (H) Spring 2026

Lecture 3 凸函数、梯度算法 — 2026.03.20

Instructor: 郦旭东

Scribe: 路人甲

目录

1 凸集与凸函数 **1**

2 梯度方法 **3**

2.1 最速下降法 3

2.2 线搜索 4

1 凸集与凸函数

首先，我们回顾一下凸集的定义。

1.1 定义 凸集

从内部视角来看，对于集合 C，如果我们从中任取两点，它们连线上的所有点也都包含在集合内，即 $\forall x, y \in C$，都有 $\lambda x + (1-\lambda)y \in C$，$\forall \lambda \in [0,1]$，那么 C 就是一个凸集。
从外部视角来看，一个凸集也可以被看作是由无穷多个半平面相切所围成的区域。作为凸集的一个特例，多面体 $P = \{x : Ax \leq b\}$ 则是具有有限个半平面交集所构成的凸集。

接下来，我们考虑凸函数。

1.2 定义 凸函数

一个函数 $f(x)$ 是凸函数，当且仅当其上方图（Epigraph） $\mathrm{epi}_f = \{(x,t) : f(x) \leq t\}$ 是一个凸集。这等价于对于定义域 Ω 内的任意两点 $x, y \in \Omega$，满足如下不等式：

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda) f(y)$$

直观地说，这意味着凸函数图像上任意两点之间的割线，始终位于函数图像的上方。

1.3 定理

假设 $\Omega \subseteq \mathbb{R}^n$ 是一个开凸集，且函数 $f : \Omega \to \mathbb{R}$ 是可微的。 f 是凸函数当且仅当

$$f(x) \geq f(y) + \langle \nabla f(y), x - y \rangle, \forall x, y \in \Omega.$$

证明. **充分性**：我们构造一个辅助函数，将多维问题转化为一维问题。定义：

$$\varphi(t) = f(x + t(y - x)), \quad t \in [0, 1]$$

为简化记号，我们设 $x_t = x + t(y - x)$. 于是 $\varphi(t) = f(x_t)$. 由方向导数的定义，$\varphi(t)$ 关于 t 的导数为

$$\varphi'(t) = \langle \nabla f(x_t), y - x \rangle.$$

利用假设中的一阶不等式，我们将 y 和 x_t 代入可得：

$$f(y) \geq f(x_t) + \langle \nabla f(x_t), y - x_t \rangle = \varphi(t) + (1 - t)\varphi'(t)$$

同理，我们将 x 和 x_t 代入，利用链式法则可以得到：

$$f(x) \geq \varphi(t) - t\varphi'(t)$$

显然，将上述第一个不等式乘以 t，第二个不等式乘以 $(1 - t)$ 并相加即可消去导数项：

$$tf(y) + (1 - t)f(x) \geq \varphi(t) = f(x + t(y - x)).$$

这恰好是凸函数的定义，因此充分性得证。

必要性：已知 f 是凸函数，根据定义有： $f(x + \lambda(y - x)) \leq (1 - \lambda)f(x) + \lambda f(y)$. f 在 $y - x$ 方向上的导数就是它在这个方向上的一阶差商。为构造出导数项，我们对其进行移项变形，考察函数值在 x 处沿着 $y - x$ 方向的增量：

$$\lim_{\lambda \to 0} \frac{f(x + \lambda(y - x)) - f(x)}{\lambda} = \langle \nabla f(x), y - x \rangle \leq f(y) - f(x)$$

这就证明了一阶条件必然成立。 □

这一刻画具有极强的直观几何意义：可微凸函数的图像始终位于其任意一点的切平面之上。

那么定义凸集、凸函数能给我们带来什么好处呢？在上节课我们提到，梯度为零是局部最优解的必要条件而非充分条件。对于非凸函数，我们在一个点 x_* 处可能满足 $\nabla f(x_*) = 0$，但它可能并不是一个局部极小值，我们还需要进一步验证二阶条件。但是，由定理 1.3 显然可知，凸性加上 $\nabla f(x_*) = 0$ 意味着 $f(x) \geq f(x_*) + 0$，即推导出 x_* 是全局最优解。这无疑是非常强大的性质：对于凸函数，我们只需要找到一个满足一阶条件的点，就能保证它是全局最优解。这也是为什么我们

在优化问题中非常关注凸函数。当然，尽管我们有着如此偏好，但本课程并不会局限于凸优化。在以后讨论的大多数问题中，我们并不会对目标函数的凸性做出任何假设。

为了在实际中找到这个极小值，我们通常采用算法 1 中的迭代框架：

Algorithm 1: 优化算法框架

Input: 初始点 x^0，容差 ε

Output: 一个近似最优解 x^*

while $\|\nabla(x^k)\| > \varepsilon$ **do**

  更新 $x^{k+1} = g(x^k)$

return x^k

我们主要的目标是设计一个合适的更新函数 g，使得梯度的范数能够逐渐下降，直至收敛到一个近似最优解。有三种常见的设计：

1. 零阶方法：仅使用函数值信息来指导搜索方向和步长的选择。
2. 一阶方法：利用梯度信息来指导搜索方向和步长的选择，亦称为梯度方法。
3. 二阶方法：利用 Hessian 矩阵（即二阶导数信息）来指导搜索方向和步长的选择，常见的二阶方法包括牛顿法和拟牛顿法。

2 梯度方法

在这类迭代方法中，梯度方法是最为核心的一类。其迭代格式为：

$$x^{k+1} = x^k + \alpha_k d^k$$

其中，α_k 是步长，d^k 是搜索方向。要保证函数值下降，对方向的要求非常简单，只需满足 $\langle d^k, \nabla f(x^k)\rangle < 0$：

$$f(x^k + \alpha_k d^k) = f(x^k) + \alpha_k \langle \nabla f(x^k), d^k\rangle + o(\alpha_k) < f(x^k)$$

2.1 最速下降法

在实际操作中，最常见的选择是直接取负梯度方向，即 $d^k = -\nabla f(x^k)$.

那么，为什么负梯度方向是“最常见”的选择呢？我们不妨从贪心策略的角度来思考。我们希望在给定的单位步长内，函数值下降得最快。这等价于求解如下的优化问题：

$$\begin{aligned} \min_{d} \quad & \langle \nabla f(x^k), d\rangle \\ \text{s.t.} \quad & \|d\| = 1 \end{aligned}$$

由 Cauchy–Schwarz 不等式，我们知道 $\langle \nabla f(x^k), d\rangle \geq -\|\nabla f(x^k)\|$。当且仅当 $d^k = -\nabla f(x^k)/\|\nabla f(x^k)\|$ 时等号成立。这说明负梯度方向正是局部下降最快的方向。

如果我们不仅在方向上贪心，在步长上也贪心，即采用精确线搜索找到该方向上的最小值：

$$\alpha_k = \arg\min_{\alpha>0} f(x^k - \alpha\nabla f(x^k))$$

这便构成了最速下降法。很可惜的是，这种看似完美的“全盘贪心”策略在实际应用中会带来问题——它会导致迭代轨迹出现严重的“锯齿现象”（zig-zaging）。所谓的锯齿现象，指的是在迭代过程中，连续的搜索方向相互垂直，导致我们在曲谷中来回震荡，无法直接指向最优解。

2.1 命题

由最速下降法产生的连续两个迭代点 x^{k+1} 和 x^k 满足

$$\langle \nabla f(x^{k+1}), \nabla f(x^k)\rangle = 0.$$

证明. 根据最速下降法的定义，我们有

$$\begin{aligned} x^{k+1} &= x^k - \alpha_k \nabla f(x^k) \\ \alpha_k &= \arg\min_{\alpha>0} f(x^k - \alpha\nabla f(x^k)) \end{aligned}$$

我们定义一维函数 $\varphi(t) = f(x^k - t\nabla f(x^k))$。求导得到：

$$\varphi'(t) = -\langle \nabla f(x^k - t\nabla f(x^k)), \nabla f(x^k)\rangle$$

由于 α_k 是 $\varphi(t)$ 的极小值点，因此必有一阶最优性条件：

$$\varphi'(\alpha_k) = -\langle \nabla f(x^{k+1}), \nabla f(x^k)\rangle = 0.$$

□

这种“锯齿现象”在等高线比较扁平的函数中尤为明显，如图 1 所示。

2.2 线搜索

既然精确线搜索会带来锯齿现象，且计算代价过高，那么在实际工程中我们应该怎么办呢？实践中，我们通常采用非精确线搜索，最著名的标准便是 Armijo–Wolfe 准则。

首先，我们引入 Armijo 准则（充分下降条件）。给定参数 $c_1 \in (0,1)$，步长 α_k 需要满足：

$$f(x^k + \alpha_k d^k) \leq f(x^k) + c_1\alpha_k\langle \nabla f(x^k), d^k\rangle$$

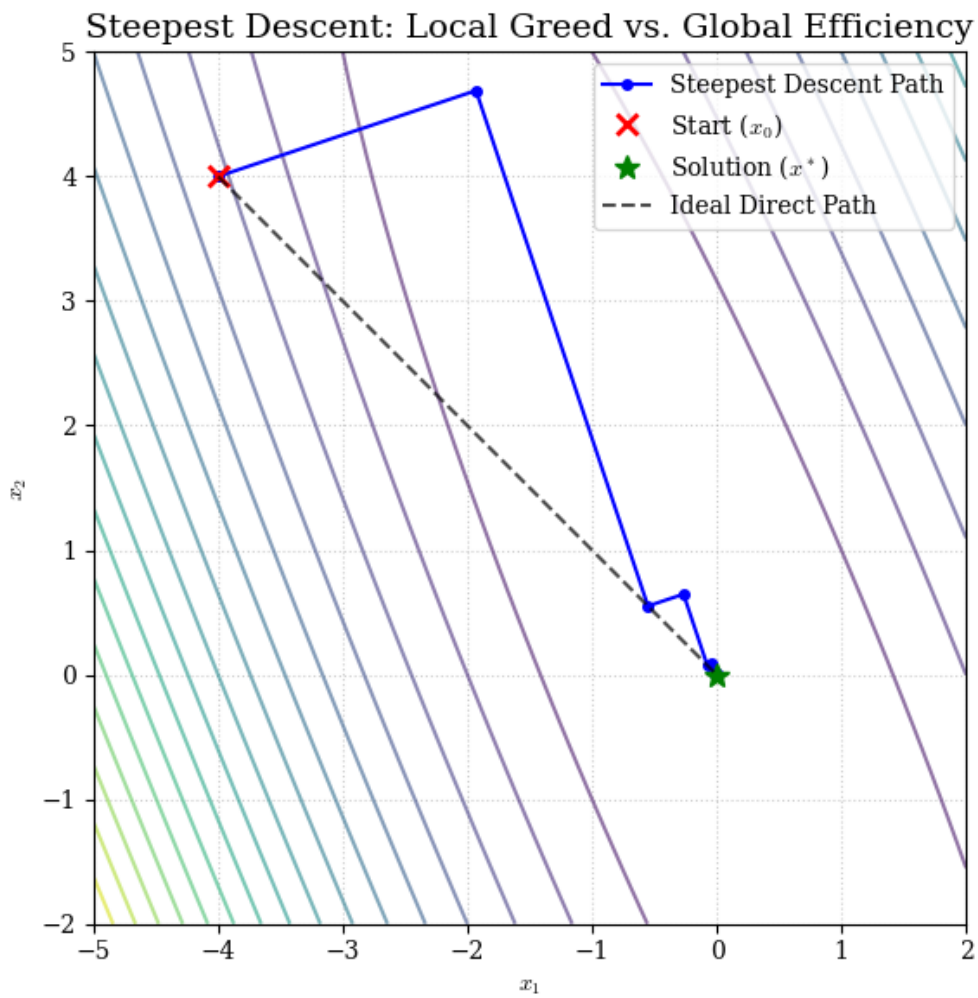

图 1: 最速下降法的锯齿现象

为了更好地分析线搜索，我们定义一维截面函数 $\phi(\alpha) = f(x^k + \alpha d^k)$。其导数为 $\phi'(\alpha) = \langle \nabla f(x^k + \alpha d^k), d^k \rangle$. 在初始点处有 $\phi'(0) = \langle \nabla f(x^k), d^k \rangle < 0$. 同时，我们将 Armijo 准则的右端项定义为一条直线 $l(\alpha) = f(x^k) + c_1 \langle \nabla f(x^k), d^k \rangle \alpha$.

现在，我们来验证满足 Armijo 准则的步长的存在性。

2.2 命题

满足 Armijo 准则的步长是必然存在的。

证明. 我们利用 Taylor 展开来证明其存在性。对于充分小的 α，我们有：

$$\begin{aligned}\phi(\alpha) &= \phi(0) + \phi'(0)\alpha + o(\alpha) \\ &= f(x^k) + \langle \nabla f(x^k), d^k \rangle \alpha + o(\alpha)\end{aligned}$$

由于 $c_1 \in (0,1)$，当 $\alpha \to 0$ 时，高阶无穷小被忽略，显然有：存在 $\bar{\alpha} \in (0,1)$，使得

$$\phi(\alpha) < l(\alpha). \quad \forall \alpha \in (0, \bar{\alpha}]$$

这说明只要步长足够小，Armijo 准则总是可以实现的。 □

基于这个命题，我们可以利用回溯法来寻找步长：初始时设定一个较大的步长（如 $\alpha_0 = 1$），如果不满足 Armijo 准则，则不断缩小（例如 $\alpha_k = \frac{1}{2}\alpha_{k-1}$）。然而，Armijo 准则存在一个缺陷：它可能导致步长过小。根据我们的分析，我们倾向于选择小步长来保证函数值的下降，但过小的步长会导致迭代效率下降，迭代次数变多。

为了解决这个问题，我们引入 Wolfe 准则。给定参数 $0 < c_1 < c_2 < 1$，除了 Armijo 准则外，我们还需要补充曲率条件：

1. $f(x^k + \alpha_k d^k) \leq f(x^k) + c_1 \alpha \langle \nabla f(x^k), d^k \rangle$
2. $\nabla f(x^k + \alpha_k d^k)^\top d^k \geq c_2 \nabla f(x^k)^\top d^k$

此外，还有一个条件被称为强 Wolfe 准则：

$$|\nabla f(x^k + \alpha_k d^k)^\top d^k| \leq c_2 \nabla f(x^k)^\top d^k.$$

那么，为什么我们引入了第二个条件就能够有效地避免步长过小的问题呢？为方便分析，我们不妨定义

$$\begin{aligned}\phi(\alpha) &= f(x^k + \alpha d^k) \\ l(\alpha) &= f(x^k) + c_1 \langle \nabla f(x^k), d^k \rangle \alpha\end{aligned}$$

于是，Wolfe 的两个条件可以等价地写为

$$\phi(\alpha) \le l(\alpha)$$
$$\phi'(\alpha) \ge c_2\phi'(0)$$

我们知道，在 0 附近，$\phi'(\alpha) \to \phi'(0) < c_2\phi'(0)$，因此我们需要步子迈大一些，才有可能满足第二个条件。

那么，满足这两个条件的 α 真的存在吗？从直觉上来说，很可能不存在。考虑一个极端的例子：$\phi(\alpha) = -x$. 由于 $c_2 < 1$，$\phi'(\alpha)$ 不可能小于 $c_2\phi'(0)$. 更一般地，如果 $\phi(\alpha)$ 无界，我们可能也找不到满足 Wolfe 准则的步长。但倘若 $\phi(\alpha)$ 有下界，我们就可以证明满足 Wolfe 准则的步长一定存在。

2.3 定理

假设目标函数**有下界**，即 $\phi(\alpha) = f(x^k + \alpha d^k)$ 在 $[0, +\infty)$ 上是有界的，则满足 Wolfe 准则的步长一定存在。

证明. 我们构造辅助函数：

$$\psi(\alpha) = \phi(\alpha) - \phi(0) - c_1\alpha\phi'(0).$$

显然 $\psi(0) = 0$，且由于 $\phi(\alpha)$ 有下界，必然有 $\psi(+\infty) = +\infty$.

我们计算它在 0 处的导数：

$$\psi'(0) = (1 - c_1)\phi'(0) < 0.$$

因为函数从 0 处开始下降，而最终趋于 $+\infty$，所以必然存在某个点 $\bar{\alpha}$，使得 $\psi(\bar{\alpha}) < 0$.

由 Rolle 定理，ψ 在 $(0, \bar{\alpha})$ 内至少存在一个点 α^*，使得 $\psi'(\alpha^*) = 0$，即

$$\phi'(\alpha^*) = c_1\phi'(0) > c_2\phi'(0).$$

同时，$\psi(\alpha^*) < 0$ 也意味着 $\phi(\alpha^*) < l(\alpha^*)$. 于是两个条件都得到了满足。 □

这一分析告诉我们，只要目标函数有下界，通过 Armijo–Wolfe 准则我们就既能保证函数值的充分下降，又能确保我们走出的步距足够大。然而问题是：我们究竟该如何搜索这样的步长？刚才我们提到的回溯法显然是不可行的，因为我们无法保证我们把搜索区间缩小后，能不能找得到满足 Wolfe 准则的步长：具体地，假如我们回溯到 1/2，那么下一步我们究竟是要在 $[0, 1/2]$ 区间内搜索，还是在 $[1/2, +\infty)$ 区间内搜索呢？我们是无法预先知道满足 Wolfe 准则的步长所在的区间的。这就导致回溯法在这里必然失败。

假设我们已知 $[1/2, 1]$ 区间满足 Armijo 准则。我们先检查端点 1 是否满足 Wolfe 准则。如果满足，我们就直接选取 1 作为步长；如果不满足，我们再检查 1/2. 如果左端点 1/2 满足 Wolfe 准则，就取 1/2 作为步长；如果 1/2 不满足 Wolfe 准则，接下来我们有两种可能的办法：

1. 将搜索区间缩小为 $[3/4, 1]$；
2. 将搜索区间放大为 $[1/2, 3/2]$.

无论最终我们选择了哪种策略，下一步我们都可以继续按照同样的方法来搜索满足 Wolfe 准则的步长。可以发现，Wolfe 准则的搜索策略比 Armijo 复杂得多。`scipy.optimize.line_search` 封装了搜索 Wolfe 步长的函数。

> 我知道这里面发生了什么，但我嫌它太繁琐，这个时候就应当放心地、大胆地调包。
>
> —— 邵美悦

除 Armijo–Wolfe 外，我们还有其他的步长选择策略，例如：

1. 非单调线搜索：允许函数值在某些迭代中不下降，但要求在一定的迭代窗口内函数值总体上是下降的。具体地

$$f(x^k + \alpha_k d^k) \leq \max_{0 \leq i \leq M} \left\{ f(x^{k-i}) \right\} + c_1 \alpha_k \langle \nabla f(x^k), d^k \rangle$$

2. 信赖域：先确定“我能走多远”（划定一个半径，即“信赖域”），然后再在这个安全范围内寻找“最优的方向和步长”。

请注意，这两种方法并不一定比普通的线搜索来得好。无论采用哪种搜索方法，我们的核心思路都是贪心：先确定一个搜索方向，然后确定步长；或者先确定一个步长，然后在这个步长范围内寻找一个合适的搜索方向。贪心是有代价的，最合理的策略应该是步长和方向同时确定。但很遗憾的是，这样的策略在传统意义上是很难实现的。但近年来随着 AI 的发展，对于一些固定的模型，我们或许可以利用 AI 同时选择步长和方向，这也是一个非常有前景的研究方向。